UNIVERSIDAD AUTÓNOMA DE CHIRIQUÍ

Facultad de Economía - Escuela de Ciencias Computacionales
Licenciatura en Gestión de Tecnología de la Información / Programación y Redes

---

Laboratorio N.º 3

Investigación Aplicada de Patrones de Comunicación

Asignatura: Programación V (PROG 309)

Docente: Alexis Flores Franco

Estudiantes:

Cristian Valdés

Samuel Suira

Dereck Martínez

David, Chiriquí

1. Introducción

El presente laboratorio tiene un carácter investigativo. Su propósito es reconocer, en un sistema real de la industria, uno de los tres patrones de comunicación estudiados en la Semana 3 del eje temático *Introducción a los Patrones Arquitecturales*: el patrón de intermediario (*broker*), el patrón de igual a igual (P2P) y el patrón de bus de eventos.

Nuestro equipo seleccionó el **patrón de bus de eventos** y, como caso real, la **arquitectura orientada a eventos de Netflix**, cuya columna vertebral es la plataforma de mensajería distribuida **Apache Kafka** (integrada en su canal de datos interno conocido como *Keystone*). Se eligió este caso porque es un sistema de uso masivo que todos los integrantes conocemos como usuarios, porque su arquitectura está ampliamente documentada por la propia empresa en su blog técnico y porque ilustra con claridad cómo el desacoplamiento entre componentes, que es el problema central que los tres patrones resuelven de formas distintas, se logra mediante publicación y suscripción de eventos.

El documento sigue el orden de las tres actividades de la guía: la selección e investigación del caso (Actividad 1) y el diagrama de componentes con su ficha técnica (Actividad 2), seguidas de una comparación entre los tres patrones. Finalmente se responden las preguntas de análisis y se presentan las conclusiones del equipo.

2. Marco Teórico

2.1 Los tres patrones de comunicación

Patrón	Idea central	Cómo resuelve el acoplamiento
Intermediario (broker)	Un componente central recibe las peticiones de los clientes y las redirige al servicio adecuado, ocultando la ubicación y la implementación de los servicios.	El cliente conoce solo al broker; nunca al servicio. El broker mantiene el registro de servicios.
Igual a igual (P2P)	Todos los nodos tienen el mismo rol y actúan a la vez como cliente y servidor; no existe autoridad central.	No hay un punto único; la responsabilidad se distribuye entre todos los nodos.
Bus de eventos	Los componentes no se llaman entre sí: publican eventos en un bus y otros componentes se suscriben a los eventos que les interesan.	Productores y consumidores no se conocen; solo comparten el contrato del evento y el bus.

2.2 El diagrama de componentes

Un diagrama de componentes (UML) representa los elementos de software de un sistema, como módulos, servicios o aplicaciones, y las relaciones o interfaces de comunicación entre ellos, sin describir el detalle interno de cada uno. Cada componente se dibuja como un rectángulo etiquetado con su nombre y, opcionalmente, con su rol dentro del sistema; las flechas indican el sentido del flujo de comunicación o las dependencias. Es la herramienta adecuada para representar visualmente un patrón de comunicación, ya que el interés está precisamente en *quién habla con quién y a través de qué*, no en cómo está programado cada componente.

3. Objetivos

3.1 Objetivo general

Investigar y analizar un caso real de la industria que aplique el patrón de bus de eventos, identificando los componentes que intervienen y la forma en que se comunican.

3.2 Objetivos específicos

- Seleccionar, en equipo, un caso real (Netflix / Apache Kafka) que utilice el patrón de bus de eventos.
- Elaborar un diagrama de componentes que muestre la comunicación entre los productores, el bus y los consumidores de eventos.
- Redactar una ficha técnica breve del caso investigado con sus fuentes.
- Contrastar el patrón elegido con los patrones broker y P2P, identificando en qué situaciones conviene usar cada uno.

4. Actividad 1 — Selección e Investigación del Caso

4.1 Patrón elegido y justificación

El equipo eligió el **patrón de bus de eventos**. La razón principal es que este patrón representa la forma dominante en que las grandes plataformas de internet actuales integran decenas o cientos de microservicios: en lugar de que cada servicio llame directamente a los demás, cada uno publica *hechos que ya ocurrieron* (eventos) y deja que quien los necesite reaccione. Esto lo convierte en el patrón más rico para analizar el problema del acoplamiento, que es el hilo conductor de la Semana 3.

4.2 El caso: Netflix y su arquitectura orientada a eventos con Apache Kafka

Netflix es un servicio de *streaming* con más de 300 millones de suscriptores en el mundo, cuya plataforma está construida sobre cientos de microservicios desplegados en la nube de Amazon Web Services. Cada acción del usuario (iniciar sesión, buscar, reproducir, pausar, cambiar de perfil, calificar un título) y cada evento interno del sistema (errores, métricas de latencia, cambios de catálogo) se convierte en un **evento** que se publica en un bus de eventos implementado con **Apache Kafka**.

Netflix documentó esta arquitectura en su blog técnico bajo el nombre de **Keystone**, su canal de datos en tiempo real. Según las cifras publicadas por la propia empresa, en 2016 el sistema procesaba más de 500 mil millones de eventos por día (alrededor de 1,3 petabytes) y para 2018 superaba el billón de eventos diarios en horas pico. Kafka funciona en este caso como el bus central: recibe los eventos de los productores, los organiza en *topics* (canales temáticos) y los entrega a todos los consumidores suscritos, de forma asíncrona y durable.

4.3 Problema que resuelve el patrón en este caso

Antes de adoptar un bus de eventos, un servicio como el de reproducción tendría que llamar directamente al servicio de recomendaciones, al de analítica, al de monitoreo y al de historial cada

vez que un usuario iniciara un video. Ese modelo de comunicación directa genera tres problemas que Netflix necesitaba resolver a su escala:

1. **Acoplamiento fuerte:** el productor debía conocer a todos sus destinatarios, sus direcciones y sus interfaces; agregar un nuevo consumidor implicaba modificar el productor.
2. **Fragilidad:** si uno de los destinatarios estaba caído o lento, la operación del usuario se veía afectada, aun cuando ese destinatario (por ejemplo, la analítica) no fuera esencial para reproducir el video.
3. **Escalabilidad limitada:** con cientos de servicios, el número de conexiones directas crece de forma cuadrática y se vuelve imposible de administrar.

El bus de eventos resuelve los tres: el servicio de reproducción publica un único evento *PlaybackStarted* en el bus y termina su trabajo; los consumidores interesados lo leen a su propio ritmo, se pueden agregar o eliminar sin tocar al productor y, si uno falla, los demás siguen funcionando porque los eventos quedan almacenados en Kafka hasta que sean consumidos.

4.4 Componentes que intervienen

Componente	Rol en el patrón	Función en el sistema de Netflix
Aplicaciones cliente (TV, móvil, web)	Productor	Generan eventos de interacción del usuario: reproducción, pausa, búsqueda, navegación por el catálogo.
Servicio de Reproducción (Playback)	Productor	Publica eventos del ciclo de vida de cada sesión de video: inicio, calidad de *bitrate*, rebúfer, fin.
Servicio de Cuentas y Facturación	Productor	Publica eventos de negocio: alta de suscriptor, cambio de plan, pago realizado.
Apache Kafka (bus de eventos)	**Bus**	Recibe, almacena de forma durable y distribuye los eventos organizados en *topics*. Es el único punto de contacto entre productores y consumidores.
Servicio de Recomendaciones	Consumidor	Se suscribe a los eventos de reproducción para actualizar el perfil de gustos del usuario y la página de inicio personalizada.
Procesamiento en tiempo real (Apache Flink)	Consumidor	Calcula métricas al instante (reproducciones por segundo, tasa de errores) y detecta anomalías.
Almacén analítico (Amazon S3 / Hive)	Consumidor	Persiste todos los eventos para análisis histórico, reportes y entrenamiento de modelos.
Búsqueda e indexación (Elasticsearch)	Consumidor	Indexa eventos y registros para búsquedas operativas y depuración.
Monitoreo y alertas	Consumidor	Observa eventos de error y latencia para alertar al equipo de operaciones.

4.5 Cómo se comunican los componentes

La comunicación sigue el modelo **publicar/suscribir** en cuatro pasos:

1. **Publicación:** un productor construye un evento (por ejemplo, *PlaybackStarted* con el identificador del usuario, el título y la marca de tiempo) y lo envía al *topic* correspondiente de Kafka. El productor no espera respuesta ni sabe quién lo leerá.

2. **Almacenamiento:** Kafka anota el evento al final del *topic*, lo replica en varios servidores (*brokers* del clúster) y lo conserva por el tiempo configurado.
3. **Suscripción:** cada consumidor está suscrito a uno o varios *topics* y lee los eventos en orden, llevando su propio registro de posición (*offset*), de modo que varios consumidores leen el mismo evento de forma independiente.
4. **Reacción:** cada consumidor procesa el evento según su responsabilidad: recomendar, medir, almacenar o alertar.

La comunicación es, por tanto, **asíncrona, indirecta y de uno a muchos**: un solo evento publicado puede desencadenar reacciones en cinco o más servicios que nunca se comunican entre sí.

5. Actividad 2 — Diagrama de Componentes

El siguiente diagrama de componentes representa el caso investigado. Cada rectángulo es un componente de software etiquetado con su nombre y su rol dentro del patrón (productor, bus o consumidor). Las flechas continuas indican **publicación** de eventos hacia el bus y las flechas discontinuas indican **suscripción** y entrega de eventos desde el bus hacia los consumidores. Los productores y los consumidores no tienen ninguna conexión directa entre sí: toda la comunicación pasa por el bus.

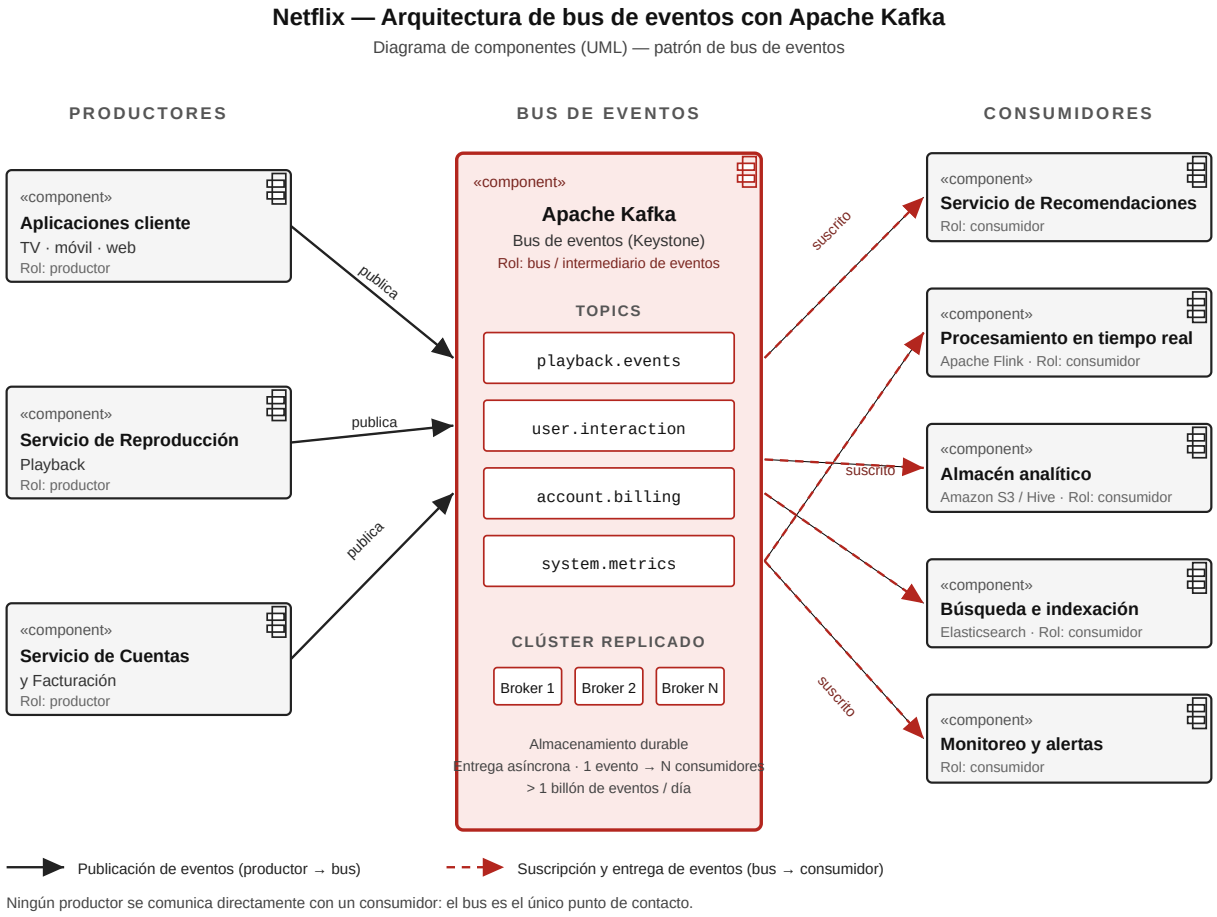


Figura 1. Diagrama de componentes del patrón de bus de eventos aplicado a la plataforma de Netflix (Apache Kafka / Keystone). Elaboración propia del equipo.

5.1 Lectura del diagrama

- **Columna izquierda (productores):** las aplicaciones cliente, el servicio de reproducción y el servicio de cuentas generan eventos y los publican en el *topic* que corresponde a su dominio. Ninguno espera respuesta.

- **Columna central (bus):** Apache Kafka organiza los eventos en *topics* (`playback.events`, `user.interaction`, `account.billing`, `system.metrics`), los almacena de forma durable en un clúster de varios servidores replicados y los entrega a todos los suscriptores.
- **Columna derecha (consumidores):** cinco servicios independientes se suscriben a los *topics* que necesitan. Nótese que `playback.events` alimenta simultáneamente a Recomendaciones y al Almacén analítico, y que `system.metrics` alimenta tanto al Procesamiento en tiempo real como al Monitoreo: un solo evento publicado produce varias reacciones.

5.2 Ficha técnica del caso

Nombre del caso / empresa	Netflix, Inc. — Plataforma de *streaming* de video. Canal de datos en tiempo real **Keystone**.
Patrón aplicado	**Bus de eventos** (arquitectura orientada a eventos, modelo publicar/suscribir).
Tecnología que implementa el bus	Apache Kafka (plataforma de *streaming* distribuida, de código abierto, originada en LinkedIn en 2011).
Componentes principales	Productores: aplicaciones cliente, servicio de reproducción, servicio de cuentas. Bus: clúster de Kafka con *topics*. Consumidores: recomendaciones, Apache Flink, Amazon S3/Hive, Elasticsearch, monitoreo.
Escala documentada	Más de 500 mil millones de eventos por día en 2016; más de un billón de eventos por día en 2018, según el blog técnico de Netflix.
Problema que resuelve	Desacoplar cientos de microservicios que necesitan reaccionar a las mismas acciones del usuario, sin comunicación directa entre ellos y sin que la caída de uno afecte a los demás.
Fuentes de la investigación	Netflix Technology Blog (2016), *Evolution of the Netflix Data Pipeline*; Netflix Technology Blog (2018), *Keystone Real-time Stream Processing Platform*; Apache Software Foundation, *Apache Kafka Documentation*; Richards y Ford (2020), *Fundamentals of Software Architecture*. Referencias completas en la sección 9.

6. Comparación entre los Tres Patrones

Como cierre de la investigación, el equipo contrastó el patrón elegido con los otros dos patrones estudiados en la semana. El siguiente cuadro resume las diferencias principales y, en particular,

en qué situaciones conviene usar cada uno.

Criterio	Broker	P2P	Bus de eventos (nuestro caso)
¿Quién conoce a quién?	El cliente conoce al broker; el broker conoce a los servicios.	Cada nodo conoce a algunos vecinos; todos son iguales.	Nadie conoce a nadie: solo se conoce el bus y el formato del evento.
Estilo de comunicación	Petición/respuesta, normalmente sincrónica.	Intercambio directo entre pares.	Publicar/suscribir, asincrónica, uno a muchos.
Punto único de falla	El broker, salvo que esté replicado.	No existe.	El bus, mitigado con clúster replicado (Netflix).
Facilidad para agregar componentes	Registrar el servicio en el broker.	Un nodo nuevo se une a la red.	Un consumidor nuevo se suscribe; el productor no cambia.
¿Para qué se usaría?	Cuando muchos clientes necesitan invocar servicios distribuidos sin conocer su ubicación y esperan una respuesta; por ejemplo, integrar sistemas heterogéneos de una empresa mediante un middleware.	Cuando se busca compartir recursos o datos entre muchos participantes sin depender de un servidor central, tolerando que los nodos entren y salgan; por ejemplo, distribución de archivos o registros descentralizados.	Cuando un mismo hecho debe ser procesado por varios servicios independientes de forma asíncrona y sin que el emisor los conozca; por ejemplo, plataformas de microservicios, analítica en tiempo real y sistemas de notificaciones.
Caso típico	Middleware empresarial (ESB), CORBA.	BitTorrent, blockchain.	Netflix, LinkedIn, Uber (Kafka); sistemas de notificaciones.

7. Preguntas de Análisis

1. ¿Por qué el patrón elegido por su equipo es más adecuado para el caso investigado que los otros dos patrones vistos esta semana?

Porque la necesidad central de Netflix es que **un mismo hecho** (un usuario inició un video) sea aprovechado por **muchos servicios distintos** (recomendaciones, analítica, monitoreo, búsqueda) sin que el servicio que lo genera tenga que esperar ni conocer a ninguno de ellos. El patrón broker no encaja bien porque está pensado para petición/respuesta: el cliente pide un servicio concreto y espera un resultado; en Netflix el productor no quiere ningún resultado, solo quiere informar. El patrón P2P tampoco encaja porque los componentes no son iguales entre sí (una aplicación de televisor y un clúster de analítica tienen roles completamente distintos) y porque Netflix necesita un registro durable y ordenado de todos los eventos, algo que una red de pares sin centro no garantiza. El bus de eventos es el único de los tres que ofrece a la vez comunicación uno a muchos, asincronía y persistencia de los mensajes.

2. ¿Qué ocurriría si el broker o el bus de eventos del caso investigado dejara de funcionar? ¿Existe algún mecanismo de respaldo?

Si el bus dejara de funcionar por completo, los productores no podrían publicar y los consumidores dejarían de recibir eventos: se detendrían las recomendaciones en tiempo real, las métricas y las alertas, aunque la reproducción de video como tal seguiría funcionando porque no depende de una respuesta del bus. Netflix mitiga este riesgo con varios mecanismos: (a) Kafka no es un servidor único sino un **clúster de múltiples servidores** y cada *topic* está **replicado** en varios de ellos, de modo que si uno cae otro asume el liderazgo automáticamente; (b) los productores mantienen un **búfer local** y reintentan el envío cuando el bus vuelve a estar disponible; (c) los eventos ya publicados están **almacenados en disco**, por lo que un consumidor que se recupere puede reanudar desde su última posición (*offset*) sin perder datos; y (d) Netflix despliega sus clústeres en varias zonas de disponibilidad de AWS, de forma que la caída de un centro de datos no detiene el bus. En consecuencia, el bus es un punto central lógico, pero no un punto único de falla físico.

3. Si el caso investigado usa P2P, ¿cómo garantiza el sistema la seguridad o la confianza entre nodos que no se conocen entre sí?

Esta pregunta no aplica directamente a nuestro caso, ya que Netflix no usa el patrón P2P sino un bus de eventos con autoridad central. Sin embargo, el contraste es útil: en un sistema P2P como BitTorrent la confianza se logra verificando cada fragmento recibido contra un *hash* criptográfico conocido de antemano, y en blockchain mediante firmas digitales y consenso entre nodos. En el bus de eventos de Netflix el problema se resuelve de forma distinta: al existir un componente central, la confianza se centraliza en él. Kafka autentica a cada productor y consumidor (con certificados TLS o mecanismos como SASL), cifra el tránsito y aplica listas de control de acceso

que definen qué servicio puede escribir o leer en cada *topic*. Es decir, el bus de eventos cambia el problema de "confiar en desconocidos" por el de "proteger al intermediario", que es más sencillo de administrar en una empresa.

4. Retomando el desafío cognitivo de la clase teórica: ¿qué ventajas concretas observó, en el caso investigado, del patrón elegido frente a una comunicación directa entre los componentes?

- **Desacoplamiento real:** el servicio de reproducción publica un evento y no necesita saber que existen cinco consumidores. Agregar un sexto (por ejemplo, un nuevo modelo de recomendación) no requiere modificar ni redesplegar al productor.
- **Tolerancia a fallos parcial:** si el almacén analítico se cae durante una hora, el usuario sigue viendo su serie; los eventos quedan en Kafka y se procesan cuando el consumidor regresa. Con llamadas directas, esa caída habría bloqueado o ralentizado la reproducción.
- **Escalabilidad independiente:** cada consumidor se escala según su propia carga. El procesamiento en tiempo real puede tener cientos de instancias mientras el monitoreo tiene unas pocas, y ninguno afecta al otro.
- **Reducción de conexiones:** con comunicación directa, N productores hablando con M consumidores requieren N × M integraciones; con el bus se requieren solo N + M. Para cientos de servicios, la diferencia es de órdenes de magnitud.
- **Trazabilidad y reproducibilidad:** como el bus conserva los eventos, el equipo puede volver a procesar el historial para depurar un error o entrenar un nuevo modelo, algo imposible con llamadas directas que no dejan registro.

8. Conclusiones

Durante esta práctica comprobamos que el patrón de bus de eventos, presentado en la teoría de la Semana 3 como una forma de comunicación indirecta mediante publicación y suscripción, es exactamente el mecanismo que sostiene una plataforma de la escala de Netflix. El análisis del caso nos permitió ver que los tres patrones estudiados resuelven el mismo problema de fondo, el acoplamiento entre componentes, pero con estrategias distintas: el broker centraliza la petición y la respuesta, el P2P elimina el centro y el bus de eventos centraliza únicamente el transporte de hechos, dejando a cada componente libre de conocer a los demás. Aprendimos también que el diagrama de componentes es la herramienta natural para comunicar estas ideas, porque muestra con precisión quién publica, quién consume y por dónde pasa la información, sin necesidad de entrar en el código de cada servicio.

9. Referencias

Apache Software Foundation. (2024). *Apache Kafka documentation: Introduction and design*. https://kafka.apache.org/documentation/

Blancarte, O. (2020). *Introducción a la arquitectura de software: Un enfoque práctico*. Oscar Blancarte Software Architecture.

Huaman, W. C. (2018). *Los 10 patrones comunes de arquitectura de software*. Medium.

Netflix Technology Blog. (2016). *Evolution of the Netflix data pipeline*. https://netflixtechblog.com/evolution-of-the-netflix-data-pipeline-da246ca36905

Netflix Technology Blog. (2018). *Keystone real-time stream processing platform*. https://netflixtechblog.com/keystone-real-time-stream-processing-platform-a3ee651812a

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Object Management Group. (2017). *OMG Unified Modeling Language (UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1